R1.01 - Variables, Allocation Mémoire RAM, Types Primitifs et Structures Conditionnelles

Thème principal : Modèle mémoire de la variable, typage strict, pièges de l'overflow et de l'IEEE 754, algorithmique conditionnelle et évaluation paresseuse

Synthèse exécutive : Ce cours fondamental de la ressource R1.01 (Initiation au Développement) pose les bases matérielles et algorithmiques nécessaires à la programmation impérative en BUT Informatique. Il explicite le lien physique entre la mémoire vive (RAM) et l'abstraction logicielle de variable (définie par son adresse, son nom symbolique, son type et sa valeur). Le cours démontre l'importance critique du choix des types primitifs pour éviter les erreurs d'overflow (dépassement de capacité) et analyse la représentation binaire des nombres réels selon la norme IEEE 754. Enfin, il détaille le pilotage de l'exécution séquentielle au moyen de structures conditionnelles imbriquées (SI / SINON SI / SINON) tout en tirant profit de l'évaluation paresseuse (short-circuit) pour optimiser l'exécution et sécuriser le code contre les plantages.

1. Le Concept de Variable et le Modèle Mémoire (RAM)

Pour qu'un ordinateur exécute un programme, le processeur doit pouvoir lire et modifier temporairement des données en mémoire vive (RAM). La variable est l'abstraction matérielle fondamentale qui permet au développeur d'interagir avec la RAM sans manipuler directement des adresses physiques complexes (ex: 0x7FFF5FBFF05C). Une variable est caractérisée de façon indissociable par : 1) Son NOM, choisi par le développeur dans le code source ; 2) Son ADRESSE MÉMOIRE, qui localise de manière unique la case mémoire allouée par le système ; 3) Son TYPE, qui indique au compilateur le nombre d'octets à réserver (ex: 4 octets pour un 'int') et le schéma de décodage des bits ; 4) Sa VALEUR, qui est le motif binaire actuellement présent dans cet espace. Lors d'une affectation (a = 5), la valeur binaire de 5 est inscrite à l'adresse associée à 'a'. Une réaffectation (a = 12) remplace définitivement l'ancienne valeur. La mémoire RAM ne conserve aucun historique des valeurs passées.

- Variable : Abstraction matérielle représentant un emplacement réservé dans la mémoire vive (RAM).
- Les 4 propriétés d'une variable : Adresse physique (hexadécimale), Nom symbolique (identificateur), Type (taille et décodage) et Valeur (contenu binaire).
- Affectation destructive : Action d'écrire une nouvelle valeur binaire à l'adresse mémoire, écrasant et effaçant définitivement la valeur précédente.

Cas concret enseignant : *Analogie du casier dans le vestiaire de l'IUT : Chaque casier possède un numéro d'emplacement fixe et immuable (l'adresse mémoire). Le développeur colle une étiquette sur la porte avec un nom symbolique (ex: 'age'). La taille physique du casier détermine son type et sa capacité d'accueil. Placer un sac rouge dans le casier équivaut à affecter la valeur 20. Si vous le remplacez ultérieurement par un sac bleu (valeur 21), le sac rouge disparaît définitivement du casier. Si vous tentez de faire rentrer une armoire dans ce casier, le casier déborde ou casse (overflow).*

2. Types Primitifs, Représentation Binaire (IEEE 754) et Overflow

Le choix du type de donnée fixe le nombre d'octets alloués et définit la plage de valeurs représentables. Un octet (8 bits) offre 2^8 = 256 combinaisons. Un entier signé de 32 bits ('int') s'étend de -2 147 483 648 à +2 147 483 647. Si une opération arithmétique dépasse cette limite, le bit de poids fort est tronqué et la valeur 'boucle' brutalement vers la borne minimale négative : c'est le phénomène d'overflow. Pour les nombres à virgule, les ordinateurs utilisent la norme IEEE 754. De même que 1/3 n'a pas de représentation décimale finie en base 10 (0.3333...), des nombres décimaux simples comme 0.1 ou 0.2 n'ont pas de représentation binaire exacte en base 2. Ainsi, l'opération (0.1 + 0.2) produit 0.30000000000000004. Il est donc formellement proscrit de comparer deux réels avec l'opérateur '==' en examen. On utilise une comparaison par tolérance epsilon : std::abs(a - b) < 0.00001.

- Dépassement de capacité (Overflow) : Erreur survenant lors de l'affectation d'une valeur dépassant la borne maximale d'un type, provoquant un bouclage vers la limite négative.

• Norme IEEE 754 : Standard informatique d'encodage des flottants causant des imprécisions d'arrondi binaire sur les décimaux.

• Interdiction d'égalité stricte (==) sur les float : Obligation de comparer deux flottants via une différence absolue inférieure à un seuil epsilon (abs(a - b) < epsilon).

• Règle de gestion financière : Interdiction d'utiliser le type float pour des devises monétaires ; utilisation obligatoire d'entiers exprimés en centimes.

Cas concret enseignant : *Catastrophe de la fusée Ariane 5 (vol 501 en 1996) : Le lanceur s'est autodétruit 37 secondes après son décollage en raison d'une exception d'overflow. Un logiciel de bord a tenté de convertir un nombre flottant 64 bits en un entier signé 16 bits. La valeur ayant dépassé 32 767, le calculateur de bord a planté. Consigne stricte de l'enseignant : 'Ne choisissez jamais le type float pour des sommes d'argent ! Stockez toujours les prix sous forme d'entiers exprimés en centimes (ex: 1050 centimes au lieu de 10.50 €)'.*

3. Structures Conditionnelles et Évaluation Paresseuse (Short-Circuit)

Par défaut, les instructions d'un programme s'exécutent de façon séquentielle de haut en bas. Les structures conditionnelles permettent de dévier ce flux selon la valeur de vérité (VRAI ou FAUX) d'une expression booléenne. L'évaluation paresseuse est une propriété fondamentale du processeur lors du traitement des opérateurs logiques : 1) Dans une condition '(A && B)', si 'A' est FAUX, l'ensemble est obligatoirement FAUX et 'B' n'est jamais évalué. 2) Dans '(A || B)', si 'A' est VRAI, l'ensemble est obligatoirement VRAI et 'B' n'est pas évalué. Ce comportement offre une optimisation de performance et permet de protéger l'application contre des crashs d'exécution graves.

• Exécution séquentielle vs Branchement conditionnel : Altération du flux naturel d'exécution ligne par ligne au moyen d'un test booléen (SI / SINON SI / SINON).

• Opérateurs logiques : ET (&&), OU (||), NON (!).

• Évaluation paresseuse (Short-circuit evaluation) : Mécanisme interrompant l'évaluation d'une condition logique dès que son résultat final est garanti.

• Sécurisation du code : Utilisation du short-circuit pour prévenir la lecture de pointeurs nuls ou la division par zéro.

Cas concret enseignant : *Exemple d'un contrôle d'accès : Dans le test 'SI (utilisateur_existe ET mot_de_passe_correct)', si la fonction 'utilisateur_existe' renvoie FAUX, le système n'exécute pas la recherche du mot de passe en base de données. Cela économise des requêtes réseau/BDD coûteuses et évite un plantage critique du programme (de type NullPointer ou Segmentation Fault) si l'objet utilisateur n'existe pas en mémoire.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi 0.1 + 0.2 n'est-il pas exactement égal à 0.3 lorsqu'on l'affiche en programmation ?
Réponse du professeur : Parce que les ordinateurs codent les nombres à virgule en base 2 selon la norme IEEE 754. Tout comme 1/3 ne peut pas s'écrire exactement avec un nombre fini de chiffres en base 10 (0.3333...), 0.1 et 0.2 n'ont pas de représentation binaire exacte en base 2. La somme produit un léger décalage d'arrondi (0.30000000000000004). Pour tester l'égalité entre deux flottants, il faut vérifier que la valeur absolue de leur différence est inférieure à un petit seuil epsilon.

Question étudiant : Est-ce qu'une variable déclarée à l'intérieur d'un bloc SI existe toujours après la fin du bloc SINON ?
Réponse du professeur : Non, absolument pas. C'est le principe de la portée (ou scope) des variables. Une variable déclarée dans un bloc délimité par des accolades {} a une durée de vie locale à ce bloc. Dès que l'exécution franchit l'accolade fermante du bloc SI ou SINON, la variable est détruite et sa case mémoire est libérée. Si vous devez utiliser la variable après la structure conditionnelle, vous devez impérativement la déclarer avant le bloc SI.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme complet en C++ permettant de calculer et d'afficher le tarif exact d'un billet de cinéma en fonction de l'âge de l'usager et de son statut d'étudiant, selon la grille suivante : - Tarif plein général :

10.00 € - Tarif mineur (< 18 ans) : 6.00 € - Tarif étudiant majeur (>= 18 ans et statut étudiant VRAI) : 7.50 € - Tarif senior (>= 65 ans) : 5.00 € Le code doit scrupuleusement respecter la norme C++ de l'IUT d'Anglet (cartouche d'en-tête, tripartition CONSTANTES / VARIABLES / TRAITEMENTS, commentaires de flot de données, accolades Allman).

Démarche de résolution :
Étape 1 : Déclaration des constantes tarifaires et des variables d'entrée (age, estEtudiant) et de sortie (prixBillet). Étape 2 : Saisie des données au clavier avec commentaires de flot de données conformément aux exigences de l'IUT d'Anglet. Étape 3 : Ordonnancement rigoureux des branchements conditionnels du cas le plus restrictif au cas général : - Le premier test 'if (age < 18)' capte tous les mineurs. Ainsi, un mineur de 16 ans qui est aussi étudiant obtiendra directement le tarif mineur (6.00 €), qui est le plus avantageux pour lui. - Le deuxième test 'else if (age >= 65)' attribue le tarif senior (5.00 €). - Le troisième test 'else if (estEtudiant)' ne traite que les majeurs étudiants qui paieront 7.50 €. - La branche 'else' attribue le tarif plein (10.00 €) par défaut. Étape 4 : Affichage du résultat formaté à 2 décimales.

```
/*
 * En-tete du fichier
 * Nom : cinema.cpp
 * Auteur : Matéi
 * Matiere : R1.01 - Initiation au Developpement
 * Objectif : Calcul du prix d'un billet de cinema selon l'age et le statut etudiant
 */

#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    // CONSTANTES
    const float TARIF_PLEIN = 10.00f;
    const float TARIF_MINEUR = 6.00f;
    const float TARIF_ETUDIANT = 7.50f;
    const float TARIF_SENIOR = 5.00f;

    // VARIABLES
    int age;
    bool estEtudiant;
    float prixBillet;

    // TRAITEMENTS
    // (ecran) << afficher >> message demande age
    cout << "Entrez votre age : ";
    // (clavier) >> saisir >> age
    cin >> age;

    // (ecran) << afficher >> message demande statut etudiant
    cout << "Etes-vous etudiant (1 pour Oui, 0 pour Non) ? : ";
    // (clavier) >> saisir >> estEtudiant
    cin >> estEtudiant;

    // Decision tarifaire selon la priorite des conditions
    if (age < 18)
    {
        prixBillet = TARIF_MINEUR;
    }
    else if (age >= 65)
    {
        prixBillet = TARIF_SENIOR;
    }
    else if (estEtudiant)
    {
        prixBillet = TARIF_ETUDIANT;
    }
    else
    {
        prixBillet = TARIF_PLEIN;
    }

    // (ecran) << afficher >> resultat du prix du billet
    cout << fixed << setprecision(2);
    cout << "Le prix du billet est de : " << prixBillet << " euros." << endl;

    return 0;
}
```

Résultat : Pour un usager de 20 ans avec statut estEtudiant = 1 (VRAI) : - '20 < 18' est FAUX. - '20 >= 65' est FAUX. - 'estEtudiant' est VRAI -> Le prix attribué est 7.50 €. Pour un usager de 16 ans avec statut estEtudiant = 1 (VRAI) : - '16 < 18' est VRAI -> Le prix attribué est 6.00 € (la priorité du premier test garantit l'application du tarif le plus bas).

■ Auto-Évaluation Flash d'Examen

Q : Que se produit-il en mémoire vive lors d'un dépassement de capacité (overflow) sur un entier signé 32 bits ?

R : Le bit de poids fort est tronqué et la valeur boucle brutalement vers la limite minimale négative (-2 147 483 648).

Q : Pourquoi est-il interdit de comparer deux variables de type float avec l'opérateur '==' en évaluation ?

R : Parce que la représentation binaire sous la norme IEEE 754 produit des imprécisions d'arrondi. Il faut comparer si std::abs(a - b) < epsilon.

Q : Dans une condition 'if (A || B)', quel est l'effet de l'évaluation paresseuse si l'expression A est VRAIE ?

R : L'expression B n'est pas évaluée par le processeur car l'expression globale est déjà mathématiquement garantie d'être VRAIE.